Durable Agent Execution with Unknown External Outcomes

OpenBot

***Abstract*—An agent can lose its connection after an outside service has made a change. Restarting the agent does not reveal whether that change took place. This paper gives a design and analysis of durable execution for this case. It separates execution state, call attempts, owner generations, and cancellation phases. A late owner may save a known result but cannot advance current work. A call with no safe retry waits for a stored decision. Model output carries an attempt number, and observers read a durable event log. We give safety properties, proof arguments, and executable models of the reply-versus-retry race. A bounded search covers 287 states and 569 state-changing edges. Three guard removals each yield a short counterexample. The model distinguishes locally admitted sends from provider effects. It shows why an explicit retry may still cause a second effect. We also define a process fault study with an independent provider ledger. The model results do not establish reliability or speed in a running system.**

***Index Terms*—Durable execution, Unknown outcomes, Agent systems, Crash recovery**

I. Introduction

An agent asks a service to create a record. The service commits the record, but the reply is lost. The agent then stops. A new process sees an unfinished call. Sending it again may create a second record. Marking it failed may hide the first record. Marking it complete would invent a result. The correct local state is that the outcome is unknown.

An agent mixes model requests, streamed text, tool calls, human decisions, and work that may outlive its parent. Crashes, cancellation, and takeover can overlap. A model retry may produce different text. A late tool response may contain useful evidence after its worker has lost the right to change the task.

Our question is: what local contract lets an agent recover while preserving known results and the limits of its knowledge? A durable workflow engine still needs such a contract for outside calls.

This design and analysis paper defines durable records, transitions, and recovery operations. Its evidence has three levels: arguments about the stated rules, runs of a small executable model, and a plan for process tests. Each level has a separate claim. In particular, a passing model does not prove that a database implementation uses the required locks.

The contributions are a precise boundary between monotone call evidence and fenced control; a recovery rule that orders late evidence against an explicit retry; and a claim-to-test map with an outside-effect oracle. The design combines known database and workflow methods. It does not claim to invent durable replay, fencing, or idempotency. Its specific question is how those methods compose when a late reply is useful but the reply holder has lost control.

The scope is distributed execution correctness. We assume an access check decides whether a caller may act. We do not design that decision system, prove the meaning of generated programs, or compare task scheduling strategies.

II. Problem model

A. Storage, workers, and providers

The system has a transactional database, one or more workers, a durable workflow engine, and outside providers. A worker can stop at any instruction. Messages can be lost, delayed, duplicated, or delivered out of order. An old worker may keep running after a replacement takes ownership. The database provides durable commits, row locks, and uniqueness constraints. Every state-changing path must use the same lock and generation checks. Database corruption and dishonest providers are outside this model. A worker is faulty by stopping or delay, not by forging another worker's fence or a provider result. Access to evidence-save is limited to the admitted call and its reply path.

A provider may offer a read, a write with an idempotency key, or a write with no safe retry. An idempotency key is a stable request name that lets the provider recognize repeats. We assume that a provider advertised as supporting this feature applies one effect per key, rejects changed input, and keeps the result for the required recovery period. These are provider assumptions, not properties that a local call table can create.

One local send does not by itself imply one remote effect. If the network or provider repeats an unkeyed invocation, local state cannot suppress that repeat. Our no-automatic-repeat claim counts local send permits. The counter examples in the model assume that each delivered invocation increases the provider counter once. Keyed providers instead meet the stronger per-key rule above.

Read-only means a repeat does not repeat a business write. A model request may still incur cost and return different text. Hidden library retries must obey the same classification; otherwise one application send could cause several unsafe actions.

A worker scope owns connections and child fibers. Closing it waits for local release, but does not prove remote work stopped. Progress assumes scopes eventually close; safety does not require bounded remote reply time.

B. State is a statement of knowledge

A generation increases when ownership changes. A cancellation phase names one stop round. An event sequence orders committed events within one execution. An attempt names one call trial. These numbers have distinct jobs.

The state graph below has four end states: completed, failed, cancelled, and interrupted. Outcome unknown remains open for a decision. Waiting covers a known pause without an unknown unsafe effect.

Intended means input is saved without an admitted send. Sending means permission to cross the outside boundary committed; it does not prove bytes reached the provider. Completed means a result is saved, regardless of the execution's success.

C. Evidence and control are separate orders

For each call c, evidence K(c) is either absent or a saved result r. Define absent <= r and r <= r. Distinct saved results have no order. The rules only move upward in this order. A conflicting reply is a rejected operation, not a merge. This defines a statement about stored evidence; it does not say that a saved model answer is true.

Control is separate. Let g be the current owner generation and a be the selected attempt. A call result may become known while its sender no longer has generation g. Saving that result changes K(c), but it cannot change g, a, the resource checkpoint, or the event stream. A later current-owner operation may select the evidence under its own fence. A rule that treats "result known" as "caller may finish" breaks this separation.

For a trace prefix h, let S(c,h) count successful unsafe send permits for call c. Let D(c,h) name the saved choice that created c, or initial for attempt zero. The intended invariant is S(c,h) <= 1. Every later-attempt send also needs a matching D(c,h). This invariant concerns every trace prefix, including one that ends in a crash. It does not bound the number of explicitly approved attempts for one business request.

D. Durable records and complete state rules

An execution record is `(id, state, generation, phase, sequence, resource)`. A call record is `(id, execution, step, attempt, class, input, state, result, published)`. Result is a tagged value: absent or present, so a valid null result differs from no result. Command records map `(namespace, key)` to immutable input and a receipt. Decision records map call ID to an immutable answer. Outbox rows name commands; event rows have a unique `(execution, sequence)` key. Calls also have a unique `(execution, step, attempt)` key.

The table lists every allowed status change. A status change appends its state event in the same transaction. Unlisted changes are rejected. Repeating a command returns its old receipt without another change. Generation, evidence, and event updates can leave status unchanged; these are not extra transitions.

TABLE I
EXECUTION STATE GRAPH. UNKNOWN MEANS OUTCOME UNKNOWN. THE GUARDS IN THE TEXT ARE PART OF EVERY TRANSITION.

From	Permitted next states
Queued	Running, cancelling, interrupted
Running	Waiting, cancelling, unknown, completed, failed, interrupted
Waiting	Running, cancelling, failed, interrupted
Unknown	Running, cancelling, failed, interrupted
Cancelling	Cancelled, unknown, interrupted
Any end state	None

Claiming queued work enters running. Takeover of other open work preserves status and increases generation. The workflow invokes calls only while running, waits on a decision while unknown, and closes its scope while cancelling. Waiting and resuming require a named durable pause and its matching wakeup. Entering unknown requires an unresolved latest unsafe call; late evidence can arrive while that status still awaits a decision. Leaving unknown for running requires a consumed retry decision; it selects a late saved result or creates the next attempt. All business state changes require the current fence. Command admission and cancellation instead obtain the current row under lock.

Completed, failed, and cancelled require local scope release and no unresolved latest unsafe call. Completed also requires the selected results and resource checkpoint. Failure records a known local error, not inferred remote failure. Interrupted means an explicit stop or an unrecoverable local condition after scope release. It retains every unknown call and makes no claim of remote rollback. Cancelling enters unknown if release exposes an unresolved unsafe call; otherwise it may enter cancelled.

For calls, intended can only become sending. Sending can become completed or unknown. Unknown can become completed when evidence arrives. Completed has no outgoing transition; an equal result save is a no-op and a different result conflicts. A keyed send may repeat while still sending, using the same key. A read retry creates a new attempt after marking its unfinished predecessor unknown. An unsafe retry creates a new attempt only through the stored decision. No operation returns a call to intended. The latest attempt is the greatest attempt number for its step, not the call with the latest response time.

III. Relation to prior work

Birrell and Nelson make failure semantics part of the RPC interface [1]. We carry missing-response uncertainty into agent execution through attempt-tagged output and a durable decision point.

Herlihy and Wing define linearizability through an operation appearing to take effect at one point between invocation and response [2]. We use this lens for local admission and state changes. An arbitrary provider and the local database have no shared commit point.

RIFL records RPC results and detects repeats to provide exactly-once semantics in a cooperating storage system [3].

Saving results matters alongside suppressing repeats. A client cannot install that protocol inside every provider. We specify local behavior when provider-supported idempotency is absent.

Beldi logs supported operations with their effects and uses replay to recover stateful serverless workflows [4]. Its Section 4.5 gives only at-least-once execution for legacy functions outside the Beldi API. This is a close comparison: cooperation at the effect boundary changes the guarantee. Our unknown state is useful at the boundary where such cooperation is absent. The added design question is how a late reply, a new owner, and a stored retry choice can coexist without giving the late owner current control.

HTTP idempotency concerns the intended effect of repeated requests [5]. It promises neither identical returned data nor free processing. Our keyed-write contract additionally requires retained results; a method name alone does not establish it.

Temporal advises making activities idempotent because retry can execute them more than once [6]. Microsoft's durable orchestration guidance separates replayed control flow from outside I/O and requires deterministic orchestration behavior [7]. These systems establish the value of durable history. Our design makes a related boundary explicit: a workflow step can be replayable while its embedded outside write remains unsafe to repeat. A model result must be saved before a dependent local tool runs.

An outbox commits a message with a database update, then delivers it later. Duplicate delivery requires idempotent consumers [8]. This closes the database-to-engine gap, but not the database-to-provider gap for arbitrary writes.

IV. Execution method

A. Admit commands once

A command key names both a request and its input. Admission holds a command lock, compares repeated input, and creates the execution, initial events, and outgoing command in one transaction. Equal input returns the saved receipt. Different input under the same key is a conflict. Returning a saved receipt is still subject to current access checks, which are assumed here as an external predicate.

The outbox is a table of commands waiting for durable engine acceptance. Its sender uses the saved execution reference. If it stops after engine acceptance but before outbox acknowledgement, delivery may repeat. Stable IDs and engine deduplication prevent this from creating a second execution. Commands carry references to sealed records rather than full prompts or results. Outbox delivery never calls the tool provider inside the admission transaction.

Database admission accepts work durably; engine acceptance allows resumption. Neither proves an outside action occurred. A receipt exposes only admission.

B. Acquire ownership and fence local changes

To take ownership, a worker locks the execution row and increases its generation. Later state changes carry that generation. A fence is the pair of execution ID and generation. A write validates the fence while holding the row lock, within the transaction that makes the change. A check performed before the transaction would allow an old owner to write after takeover.

The current owner alone may advance the execution, commit a resource checkpoint, or publish current call completion. A resource checkpoint also locks the resource and requires its active execution ID to match. It changes the resource revision and appends a checkpoint event in one transaction. Reading a newer revision therefore cannot restore an old owner's right to write.

The fence governs local commits. There is no global linearization point that joins database admission and an arbitrary provider effect. It does not revoke a request already in a network queue. Even a send admitted just before takeover can reach the provider afterward. A provider-side fencing token could give a stronger rule, but it would require provider cooperation. This design instead preserves the admitted call and accounts for its possible effect.

C. Separate model and tool attempts

A logical step can have several attempts. Its saved input is compared on reuse. A completed attempt returns its saved result. For a lost read or model answer, a later attempt gets a new number and call ID. An event `start(step, attempt)` establishes that number before text is published. Text and tool output carry both the step name and attempt number.

Only the selected, saved complete model result may drive local calls. Text appends also lock the execution and check its generation and latest attempt. A partial answer cannot be joined to its replacement as one decision. Both attempts remain visible.

For a write with no safe retry, recovery keeps the last attempt until a saved decision permits a new one. An explicit retry creates a new attempt and retains the old unknown record. It accepts the possibility of a second external effect; it does not prove the first attempt failed.

D. Cross the outside boundary in three steps

The worker commits intent, checks state and access, commits sending, then calls the provider outside the transaction. The call ID is the provider key for keyed writes. A result is saved before dependent work proceeds; its completion events share that transaction when publication is allowed.

A storage failure after a reply retries the result save while holding that result in memory. It does not rerun the provider request. The first save is protected from ordinary cooperative cancellation. A failed save opens an interruptible wait before another try. Process death can still destroy the only local copy of the reply. Recovery then follows the saved call state and retry class.

The following pseudocode states the send and result rules. Intent creation first locks the execution, requires running and the current generation, and inserts or checks the unique call input. All operations lock execution before calls. Database reads used by guards occur under these locks.

```
send_gate(e, g, c): TRANSACTION
  lock execution e, then call c
  require e.generation = g and e.state = running
  require c = latest(e, c.step) and access(c)
  if c.result is present: return SAVED
  if c.state in {sending, unknown}
     and c.class != keyed:
    set c.state = unknown
    if c.class = unsafe:
      set e.state = unknown; ensure decision(c)
    return RECOVER
  set c.state = sending; return SEND(c.id)

save_reply(e, g, c, r): TRANSACTION
  lock execution e, then call c
  require c.state != intended
  require absent(c.result) or c.result = r
  set c.result = present(r), c.state = completed
  if e.generation != g or e.state != running:
    return EVIDENCE_ONLY
  if c != latest(e, c.step): return EVIDENCE_ONLY
  if not c.published:
    append output(c, r) and complete(c)
    set c.published = true
  return SELECTED

invoke(e, g, c):
  action = send_gate(e, g, c)
  if action != SEND: follow saved recovery rule
  r = provider(c.id, c.input) outside transaction
  retry save_reply(e, g, c, r) on storage failure
  advance only after SELECTED
```

Listing 1. The known reply is held across save retries. A missing reply takes the defined recovery path.

SAVED invokes result-save without contacting the provider. RECOVER waits for an unsafe call's decision or starts a read's next attempt. Ensuring a decision inserts retry and stop choices once, with the unknown state event. A lost reply follows this same locked recovery rule. Keyed failures remain sending. A result envelope distinguishes success from known rejection; absence means neither.

E. Preserve late results without late control

A stale owner can know something the current owner does not. Rejecting every write from that owner would throw away a valid provider reply. The result-saving operation therefore has a narrow exception to the fence rule. It may fill the result of its already-sent call. It rejects a different result if one is already saved. It cannot change the execution state or publish current output merely because it saved evidence.

Publication requires the current generation, running status, and latest attempt. A completion marker prevents repeat publication. The current owner can publish saved evidence once; an older attempt cannot replace a newer selection.

An unknown write's decision ID is its call ID. Under the execution lock, retry consumption first returns any existing next attempt, then checks for a late saved result, and otherwise inserts attempt a+1. It moves unknown to running in that transaction. Stop closes local work and enters interrupted. A repeated answer cannot change its stored value. A late old result cannot replace an existing next attempt. Lock order, rather than response timestamps, settles this race.

F. Cancel a phase, then close its scope

A new cancel command locks an open execution, increases phase from p to p+1, and commits cancelling plus signal p+1. If already cancelling, it returns the current phase. End states reject new cancel commands. Repeated commands reuse their saved phase. Work running after phase p listens only for p+1. Its scope result is keyed by execution, generation, work phase, and awaited cancel phase. The outer workflow stays alive to record release. A takeover cannot reuse an old owner's scope result. A cancellation-end commit requires both the current generation and the saved cancel phase.

After local release, the owner inspects unresolved calls. If the latest unsafe attempt may have been sent, the execution moves to outcome unknown and remains available for a decision. Otherwise it may become cancelled. A later explicit retry belongs to a later phase. Replaying the earlier cancellation command must not stop it. Earlier unknown attempts stay in history, but they do not alone block a phase that has explicitly moved to a newer attempt.

Attached children receive cancellation commands and are awaited. Detached tools retain their own execution. Neither relation permits undoing effects. Saved deadlines prevent replay from restarting a time budget.

G. Observe committed facts

Events receive increasing sequence numbers under the execution lock. An observer asks for events after its cursor. PostgreSQL notifications wake readers, but the event table is authoritative. The proposed reader also uses periodic wakeups. A bounded live buffer may drop old notifications or events; a sequence gap causes the observer to read the missing range from storage.

An observer applies events after its cursor in sequence order and groups text by attempt. Disconnect releases only the reader. Reconnect can read the stored end event. Event retention must cover the supported cursor lifetime.

V. Safety properties and arguments

We state properties of the defined protocol under the assumptions above. The arguments concern these rules, not all possible implementations. An implementation must preserve their lock boundaries and guards.

P1: one admission per command. For a namespace and command key, equal requests select one execution and changed requests conflict. The command lock orders competing requests. The first successful transaction creates the durable record. Every later request sees it or waits for the transaction. A rollback creates no accepted record. The proof also requires a uniqueness constraint so that another entry point cannot bypass the intended lock.

P2: stale owners cannot advance work. Takeover and advancing writes lock the same execution row. If a write commits first, it precedes takeover. If takeover commits first, the stored generation differs and the write fails. Late result storage is excluded from "advance" because it does not publish completion, change control state, or update the business resource. This exclusion must be enforced structurally, not left to caller convention.

P3: no automatic repeat of an unknown unsafe write. For each unsafe call c and trace prefix h, S(c,h) is at most one. Initially it is zero. Only the intended-to-sending transition can increase it. That transition leaves intended, and no rule returns

to intended. All other steps leave S unchanged. Recovery of that attempt can return a saved result or enter unknown state, but cannot pass a new unsafe send gate. A new attempt requires a saved retry decision. This proof assumes that the provider adapter sends at most once for each admitted unsafe invocation and disables hidden unsafe retries. It guarantees no silent recovery resend. At-most-once business effects would also need a provider or delivery guarantee. It is not promised across explicit retries.

P4: evidence is monotone. Once a call has a saved result, later saves either agree or conflict. They never replace it with unknown or with a different result. An old owner can therefore improve knowledge without reversing current control. The property is per call ID. Two explicitly distinct attempts can have different results and different effects.

P5: publication selects one current attempt. The completion marker, latest-attempt check, and generation check share the locked transaction. Two publishers cannot both observe an unset marker and commit separate completion groups. A stale owner cannot publish even for the same call ID. A late older attempt cannot publish after a new one has been selected. This property concerns durable publication; a network client can still receive duplicate transport frames and must use sequence numbers.

P6: cancellation does not erase uncertainty. A final cancel commit follows scope release and examines the latest unsafe attempts. If one may have reached a provider without a known result, the state remains open as outcome unknown. Hence cancelled does not assert a false remote rollback. Cancelled states local release under the recorded conditions. It permits earlier effects and cannot stop a delayed send already admitted by an old owner.

There is a simple impossibility boundary. Consider history A, where a provider commits and its reply is lost, and history B, where the request never arrives. Both leave the local database at sending with no result. A deterministic recovery process given only that database cannot distinguish them. Automatic resend gives progress in B but may duplicate the effect in A. Refusing resend avoids that duplicate but may leave B unfinished. Provider evidence may distinguish the histories. A retained deduplication key may make repeating the request safe without distinguishing them. A human retry choice does neither; it accepts the remaining risk and permits a new attempt. Durable storage alone cannot resolve the unknown fact.

Conditional progress is weaker than safety. With fair scheduling, a working database, eventual engine delivery, terminating local release, and either a known result or an available recovery decision, the local recovery step can continue. Eventual task completion also needs later outside calls to return, finite task work, and a period without further crashes. Without a decision, an unknown write can remain open indefinitely. This is an honest limit of the interface, not a claim of deadlock freedom.

VI. Worked failure traces

These are constructed traces, not measured trials. No timing values are assumed.

TABLE II
A late result saves a needless second write when it arrives before the retry decision takes effect.

Step	Event	Durable consequence
1	Owner 7 admits write	Attempt 0 becomes sending
2	Provider makes change	Reply delayed; no saved result
3	Owner 8 takes over	Unsafe call becomes unknown
4	Owner 7 receives reply	Result saved; no completion event
5	Retry decision consumed	Known attempt 0 reused
6	Owner 8 publishes	One completion group commits

In the table, ownership changes but call identity does not. At step 4, the old generation's finish operation cannot advance the execution. At step 5, the decision path reads the completed call under lock and returns to running without creating attempt 1. Replaying step 6 sees the completion marker. The provider ledger has one effect and the local log has one selected completion.

Now exchange steps 4 and 5. The retry decision creates attempt 1 while attempt 0 is still unknown. The late result for attempt 0 is saved afterward, but it cannot select current output. If attempt 1 is sent, the provider may make a second change. This outcome does not violate P3 because the second send was explicitly requested. It demonstrates why a retry decision must communicate uncertainty rather than imply that it repaired a known failure.

A second trace concerns model output. Attempt 0 emits "Create" and then loses its connection. A new owner creates attempt 1, whose answer is "Read the existing record." Joining the fragments would form a message that no provider produced. Separate start and text events preserve both histories. Only the saved complete result of attempt 1 drives local tools. A delayed complete answer for attempt 0 can be retained, but cannot replace attempt 1.

A third trace concerns cancellation and storage failure. A tool returns a known result while cancellation is pending. The first result-save attempt fails. The worker retains the reply and waits before another save. If the wait is interrupted, local release can finish while the database still records sending. The cancellation path must record outcome unknown. If the save instead commits before shutdown, recovery has the result and does not resend. Both histories are legal; only the latter has durable proof of the reply.

If the outbox sender stops after engine acceptance, redelivery reuses the execution ID. That suppresses duplicate work admission, but cannot settle an unknown provider effect.

VII. Executable model evidence

The artifact in `papers/experiments/papers_01_05.py` separates the call table from a small provider-effect ledger. Run it from the repository root with

`devenv shell -- python3 papers/experiments/run.py --seeds 30 --output .cache/e2e/papers.json`. The report records source hashes, the Python version, seed values, checks, and limits. This experiment uses no live network, SQL transaction, or language model.

The initial recorded run used seeds 0 through 29 and passed 166 assertions. Of 30 shuffled reply-versus-retry schedules, 13 put the late reply first and 17 put retry first. These are repetitions of only two distinct orders. They are not 30 distinct fault cases or a sample from production failures. No confidence interval or reliability rate follows from this count.

The reply-first order retained attempt 0 and made one outside effect. The retry-first order selected attempt 1 and made two outside effects. Both kept attempt 0′s result unchanged and published only the selected attempt. A separate stop before network delivery left the outcome unknown with no automatic resend. The automatic-resend comparison made two effects after a lost reply; the proposed path made one when no retry was chosen.

These results support the executable race model. They do not establish P1, database atomicity, event recovery, or the complete cancellation protocol. The class named `Execution` is a reference model, not imported product code. Its cancellation method directly records a phase outcome; it does not run or await an actual scope. The process study below is still needed to test those refinement steps.

A. Bounded exhaustive search and guard removal

The separate `paper_01_extended.py` model uses immutable states and breadth-first search. It has one logical step, two owners, one takeover, two attempts, one saved retry choice, and two possible non-null reply values. Send permission, provider effect, reply save, retry choice, and publication are separate steps. The provider may act before or after takeover. A reply may remain unsaved in any trace prefix. Each delivered permit has one effect, as required by the model's stated provider assumption.

The checker reads each before-state, action, and after-state independently of the action guards. It checks P2 through P5. It also requires reached examples of stale-owner evidence saves, delivery after takeover, both reply-versus-retry orders, and two effects after an explicit retry. Equal replay and rejected operations are self loops and are omitted. No rule for command admission, cancellation, model text, outbox delivery, or event reading is present in this model.

TABLE III
COMPLETED SEARCHES. EACH SEARCH EXHAUSTS ITS REACHABLE FINITE GRAPH. AN EDGE IS ONE STATE-CHANGING MODEL STEP, NOT ONE MACHINE INSTRUCTION OR FAULT.

Configuration	States	Edges	Shortest failure
Proposed rules	287	569	None in bound
No generation check	365	805	P2 in 2 steps
No completion marker	339	667	P5 in 5 steps
No unsafe-send guard	4,062	11,431	P3 in 2 steps

In the first failing configuration, takeover followed by an old-owner send permit violates P2. In the second, one send, effect, reply save, and two publications violate P5. In the third, two successive send permits for the same unsafe attempt violate P3 before any provider effect is needed. The report stores the full shortest trace for every detected property. Breadth-first search makes these traces shortest in model steps; it does not find the shortest real process schedule.

The proposed graph has maximum shortest-path depth 10. The search also allows at most two send permits and two completion groups per attempt, so a removed guard can reveal a repeat. Those caps are exploration limits, not added correctness rules. Every removed-guard graph is exhausted within the same declared caps; searches do not stop on their first failure. No conflict-preservation guard is removed, so P4 has a reachable-edge check but no separate detecting ablation. A finite pass is neither an unbounded proof nor a measurement of failure rate.

B. From model steps to implementation steps

A refinement map tells which concrete steps count as one model step. A database transaction that accepts a command maps to admission. A locked compare-and-update maps to a fence check. A call row transition maps to send permission. Provider acceptance and effect remain distinct outside steps. Saving a reply and publishing it map to separate evidence and control changes, even when one transaction can do both. Local steps that only wait or retry a failed database transaction do not change model state.

For this map to preserve P2 through P5, each implementation path must meet four obligations. All owner-driven control updates validate the generation under the same row lock. Command admission and cancellation obtain the current row under that lock instead. Every unsafe adapter invocation consumes one successful send permission. Evidence-save can fill only the named admitted call. Selection and its publication marker commit together. A test that exercises the normal path cannot establish these obligations for another entry point.

TABLE IV
A LOCAL RECOVERY RULE DEPENDS ON SAVED FACTS. THE MIDDLE TWO ROWS MUST NOT BE SPLIT BY INFORMATION AVAILABLE ONLY TO THE TEST ORACLE.

Crash point	Last durable fact	Allowed recovery for an unsafe call
Before intent commit	No admitted call	Create intent under the current fence
After intent, before send gate	Intended	Pass the locked send gate once
After send gate, before delivery	Sending; effect absent or unknown	Wait for evidence or a stored choice
After effect, before reply save	Sending; effect unknown locally	The same recovery as the prior row
After reply save, before selection	Result known	Select with current fence; do not resend
After selected publication	Result and marker known	Return saved output; do not publish twice

VIII. Proposed evaluation

A. Harness and ground truth

The full process evaluation has not been run for this paper. It should use two operating-system worker processes, real PostgreSQL, and a separate provider emulator. The emulator keeps an append-only request ledger outside the workers' database. Each entry records request ID, input hash, actual effect, and reply release. A control channel can block before acceptance, after effect commit, and before reply delivery. This makes the ambiguous histories observable to the test without exposing that oracle to the implementation.

Use three provider modes: pure read, keyed write with durable deduplication, and unkeyed write that increments a counter. The model emulator emits a chosen sequence of text fragments and a complete answer. Give replacement attempts different answers so that accidental text mixing is detectable. The harness also records process exits, transaction barriers, and observer cursors in a replayable schedule.

Inject faults before and after intent commit, sending commit, provider effect, result save, completion publication, resource checkpoint, and outbox acknowledgement. Include process kill, dropped reply, connection reset, database rollback, delayed old-owner reply, and lost notification. Force takeover while the former worker remains alive. Inject cancellation at each of these barriers. Repeat old cancellation and decision messages after a new phase begins.

B. Baselines and ablations

Define three baseline algorithms. B1 saves intent and result, but on restart sends every call without a result, regardless of retry class. B2 saves only a workflow step's final value; on replay it returns that value if present and otherwise runs the whole step, including its provider send. B3 uses the proposed records but marks every lost-result execution interrupted, including reads and keyed writes, and never resumes it. Each otherwise uses the same command admission and provider adapter. These are defined comparison algorithms, not claims about deployed products.

For B1, provider commit followed by reply loss leaves no result, so its restart rule sends again. An unkeyed counter increases twice. For B2, provider commit followed by process death before the step-value commit has the same effect. Saving step values therefore does not close the gap within a step. B3 avoids those repeats, but a lost pure read cannot resume despite having no repeated business write. These consequences follow from the stated rules before any experiment.

Ablate generation checks, attempt-tagged text, completion markers, phase-tagged cancellation, result-save retry, and the outbox separately. Retain all other mechanisms. Use the same invariant checker and publish the shortest failure schedule. A schedule that never reaches a removed check gives no evidence about it.

C. Metrics and analysis

The primary metrics are unsafe effects repeated without a retry decision, stale advancing commits, conflicting saved results, duplicate selected completions, and text containing fragments from different attempts. Each is a count of contract violations against the provider ledger and durable history. A terminal state with an unresolved latest unsafe call is also a violation, except an explicit interrupted disposition that retains the unknown evidence and makes no rollback claim.

Secondary metrics are time from takeover to progress, time spent unknown, result-save retry count, database writes per completed step, observer catch-up delay, and retained bytes per attempt. Separate unknown time waiting for a human from infrastructure delay. Report the number of unfinished executions at the observation limit; do not treat them as fast completions or silently omit them.

First enumerate all single-fault barrier schedules. Then generate paired-fault schedules and random schedules from published seeds. Run each schedule against every configuration with the same provider behavior. For a proposed stochastic phase, first state a probability law over fault sites, delay, and fault count. Use at least 1,000 independent schedule draws per provider mode and configuration, then report exact trial counts and confidence intervals for violation rates. A new seed is not a new schedule when the generator maps it to the same event order. Report distinct schedules as well as draws. A bound applies only to the stated generator; it is not a bound on production risk. With zero violations in 1,000 independent trials, the usual one-sided 95 percent upper bound is approximately three divided by 1,000, or 0.003. This is an illustrative calculation, not a result and not a proof of safety.

Use paired bootstrap intervals over complete schedule draws for differences in recovery delay. Keep all events from one draw in the same resampled unit. Report delay only for pairs that complete in both configurations, and separately report completion rates and censored runs. This prevents a design that stops difficult work from appearing faster merely by dropping it. Give medians and tail quantiles with the sample size. Analyze safety first; a configuration that gains speed by repeating unsafe effects has not improved the target contract. The bounded state model can also be exhaustively explored for two owners, two attempts, and two cancel phases. Such exploration checks that finite model only. It complements, rather than replaces, process tests and the proof arguments.

IX. Reproducible synthetic fixtures

Use integer logical time for correctness runs. Initialize one execution E as queued with generation 6, phase 0, and sequence 0. Its resource revision is 0. Empty the call, decision, event, command, and outbox tables. Set the provider counter to 0. Worker A's claim gives generation 7; worker B's takeover gives generation 8. Use step W, attempt 0, and call C0. A next attempt is C1. These fixed names remove random ID differences from replay.

Fixture F1, late evidence first. Admit C0 as an unsafe increment of one. Let the provider change its counter to 1 and hold the reply. Transfer ownership to B, which marks C0 unknown. Release A's reply and save it. Store retry for C0, then consume it twice. Expected: counter 1, only C0, result present, and no publication by A. B's result publication

creates one output and one completion, even if repeated. A decision retry therefore means permission to continue; it need not force a new send.

Fixture F2, decision first. Repeat F1 but consume retry before releasing A's reply. This creates C1 exactly once. Send C1, then release both replies in either order. Expected: counter 2, two immutable results, and only C1 selected. C0 remains audit evidence. The second effect has an explicit retry decision; the fixture must not label it an automatic-repeat violation.

Fixture F3, no network send. Stop A immediately after C0 becomes sending and before provider invocation. B must wait with C0 unknown. Expected: counter 0, no automatic second send. This tests conservative uncertainty when the outside effect did not occur, using a schedule the worker cannot distinguish from reply loss.

Fixture F4, cancel phase. With an unknown C0, accept cancellation phase 1, release the local scope, and retain unknown. Consume retry to create C1. Deliver the old phase-1 signal twice while C1 runs. Expected: neither delivery cancels the new phase. A fresh cancel uses phase 2. Its final commit must validate both generation 8 and phase 2; a delayed phase-1 completion fails.

Fixture F5, atomicity and observation. Create a completed call result, then force rollback after writing a resource revision but before appending its checkpoint event. Expected: neither change remains. Retry to commit both. Produce five events with live buffer capacity two, drop all notifications, and reconnect with cursor 1. Expected: ordered events 2 through 5 exactly once at the consumer, with no provider send caused by observation. Polling must recover events even if no later live event exposes the gap.

For every fixture, record each command, barrier release, attempted commit, committed row version, provider effect, and emitted event. The checker reconstructs all six properties from this trace. Report schema, runtime and database versions, transaction isolation, buffer capacity, polling period, seed, and hidden retry settings. The defined records and schedules determine the expected states. These fixtures are specified tests, not reported executions.

Proof obligations include common lock order, exact result decoding, retained command identity, and tools reading only selected model results. A finite model must separate provider effect from reply delivery; combining them removes the failure window under study.

X. Limits and conclusion

The design cannot settle an unknown unkeyed write without outside evidence or an explicit choice. Provider key expiry can invalidate safe replay assumptions. Call records and events therefore need a documented retention policy tied to provider guarantees. Keeping every unknown attempt forever may be costly, but deleting one cannot be treated as proof that it had no effect.

Fences protect database state; scope closure releases local resources. Neither ensures remote rollback. Durable model results can still be wrong. Version changes need stable decoding and replay rules; this paper does not supply a migration proof.

The contract retains uncertain attempts, preserves late evidence, fences current control, numbers cancellation phases, and observes committed events by cursor. Its analysis states the limits of local recovery. The bounded models test the reply-versus-retry race and detect three guard removals; they do not prove a full implementation. The proposed process evaluation uses provider effects as ground truth, without claiming exactly-once behavior across an uncooperative outside boundary.

References

[1] A. D. Birrell and B. J. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984, [Online]. Available: https://www.cs.cmu.edu/~15712/papers/birrell84.pdf

[2] M. P. Herlihy and J. M. Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990, [Online]. Available: https://www.cs.cmu.edu/~wing/publications/HerlihyWing90.pdf

[3] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, "Implementing Linearizability at Large Scale and Low Latency," in *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015. doi: 10.1145/2815400.2815416.

[4] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and Transactional Stateful Serverless Workflows," in *14th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2020, pp. 1187–1204. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zhang-haoran

[5] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[6] Temporal Technologies, "Activity Definition." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.temporal.io/activity-definition

[7] Microsoft, "Durable Orchestrator Code Constraints." Accessed: Oct. 27, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/durable-task/common/durable-task-code-constraints

[8] Amazon Web Services, "Transactional Outbox Pattern." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html